

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI-590015, KARNATAKA



Project Report
on

**DESIGN AND IMPLEMENTATION OF MEMORY MANAGEMENT
WITH DIRECT MAPPED CACHE AND WRITE THROUGH POLICY**

Submitted in partial fulfilment of the requirements for the IV semester

Mini Project Work [SoC Design &Flow] [EC272]

Bachelor of Engineering

in

Electronics and Communication Engineering

of

Visvesvaraya Technological University, Belagavi

by

Charvanya H L (1CD23EC028)

Bhavana M V (1CD23EC019)

Isheeta Pandey (1CD23EC062)

Aishwarya Sonakar (1CD23EC007)

Under the guidance of:

Mr. V.Umesh Royal

External Guide

Design and Verification Engineer

Elevium-by Nanochip

Dr. Indu K.

Assistant

Professor

ECE, CIT



Department of Electronics and Communication Engineering
CAMBRIDGE INSTITUTE OF TECHNOLOGY, BANGALORE - 560 036

2024-2025

CAMBRIDGE INSTITUTE OF TECHNOLOGY

K.R. Puram, Bangalore-560 036

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

It is Certified that **Ms. Charvanya H L, Ms. Bhavana M V, Ms. Isheetta Pandey and Ms. Aishwarya Sonakar** bearing **USN: 1CD23EC028, 1CD23EC019, 1CD23EC062 and 1CD23EC007** respectively, are bonafide students of Cambridge Institute of Technology and have completed requirements of the project entitled “Design and implementation of memory management system with direct mapped cache and write through policy” in partial fulfilment of the requirements for IV semester Bachelor of Engineering in Electronics and Communication Engineering of Visvesvaraya Technological University, Belagavi during academic year 2024-25. It is certified that all Corrections/Suggestions indicated for Internal Assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of mini project prescribed for the Bachelor of Engineering degree.

Mr. V.Umesh Royal
External Guide,
Elevium

Dr. Indu K.
Internal Guide,
ECE Dept- CITech

Dr. Shivapanchakshari T.G.
Prof. and HOD ECE,
CITech

Dr. G. Indumathi,
Principal,
CITech

External Viva:

Name of the Examiners

Signature with Date

- 1.
- 2.

DECLARATION

We, Ms. Charvanya H L, Ms. Bhavana M V, Ms. Isheetta Pandey and Ms. Aishwarya Sonakar bearing USN: 1CD23EC028, 1CD23EC019, 1CD23EC062 and 1CD23EC007 respectively, are students of IV semester, Electronics and Communication Engineering, Cambridge Institute of Technology, hereby declare that the project entitled “Design and implementation of memory management system with direct mapped cache and write through policy” has been carried out by us and submitted in partial fulfilment of the course requirements MINI PROJECT of IV semester Bachelor of Engineering in Electronics and Communication Engineering as prescribed by Visvesvaraya Technological University, Belagavi, during the academic year 2024-2025.

We also declare that, to the best of our knowledge and belief, the work reported here does not form part of any other report on the basis of which a degree or award was conferred on an earlier occasion on this by any other student.

Date: 14/05/2025

Place: Bengaluru

Charvanya H L (1CD23EC028)
Bhavana M V (1CD23EC019)
Isheetta Pandey (1CD23EC062)
Aishwarya Sonakar (1CD23EC007)

ACKNOWLEDGEMENT

I would like to place on record my deep sense of gratitude to Shri **D. K. MOHAN, Chairman**, Cambridge Group of Institutions, Bangalore, India for providing excellent Infrastructure and Academic Environment at CITECH without which this work would not have been possible.

I extremely thankful to **Dr. G. INDUMATHI, PRINCIPAL**, Cambridge Institute of Technology, Bengaluru for providing me Academic ambience and Laboratory facilities to in and everlasting motivation to carry out this work and shaping our careers.

I express my sincere gratitude to **Dr. SHIVAPANCHAKSHARI T. G** HOD, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for his stimulating guidance, continuous encouragement and motivation throughout the course of present work.

I also wish to extend my thanks to our Internal Guide **Dr. Indu K** Assistant Professor, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for their unstilted support, valuable guidance and help throughout the work.

I also wish to extend my thanks to our External Guide **Mr. V. Umesh Royal**, Elevium, Bengaluru for his critical, insightful comments, guidance and constructive suggestion and support to improve the quality of this project work.

I would like to thank all the faculty members and non-teaching staff of Dept. of ECE and Elevium team for their constant support. And I would like to thank our parents and friends for their constant moral and financial support.

Charvanya H L (1CD23EC028)
Bhavana M V (1CD23EC019)
Isheeta Pandey (1CD23EC062)
Aishwarya Sonakar (1CD23EC007)

ABSTRACT

Efficient memory management is a critical aspect of modern computing systems, directly influencing system performance, power efficiency, and responsiveness. Our project focuses on the design and implementation of a simplified memory management system featuring a direct-mapped cache architecture with a write-through policy, developed using verilog hardware description language. The goal is to simulate a small-scale yet functional model of cache memory behaviour to demonstrate fundamental concepts in computer architecture and digital system design. The system is designed to operate as an intermediary between a CPU and main memory, providing faster access to frequently used data and instructions. A direct-mapped cache structure is selected for its simplicity and speed. In this mapping scheme, each block of main memory maps to exactly one cache line, reducing the complexity of address comparison logic and ensuring a deterministic mapping. Although it has limitations in terms of conflict misses, its straightforward structure is ideal for educational and foundational design purposes. The write-through policy ensures that every write operation to the cache also updates the corresponding location in the main memory. This policy simplifies consistency management between cache and memory, as there is no risk of data becoming stale. While write-through introduces a performance trade-off due to more frequent memory writes, it eliminates the need for complex write-back logic and improves reliability in systems where data integrity is paramount.

The verilog implementation includes modules for the cache controller, tag comparator, address decoder, memory interface, and basic finite state machines (FSMs) for read and write operations. The system simulates how memory requests are handled: on a cache hit, data is retrieved from or written to the cache directly; on a cache miss, data is fetched from the main memory, and the cache is updated accordingly. The design also supports a simple testbench to simulate various access patterns and validate cache behaviour under different scenarios.

Table of Contents

Contents	Page no.
1. Introduction	1
1.1 Objective	2
1.2 Research Methodology	2-3
2. Literature Survey	4-5
2.1 Memory hierarchy and cache fundamentals	4
2.2 Cache mapping techniques	4
2.3 Write policies in cache	5
2.4 Verilog HDL for memory systems	5
2.5 Simulation and evaluation techniques	5
3. Research Gap	6
4. Block diagram	7
4.1 Flow summary	7
5. Methodology	8
6. Flow Chart /Algorithm	9
7. Software used	10
7.1 Key features	10
8. Result Analysis	11-12
9. Applications	13
9.1 Computer architecture education	13
9.2 Embedded system design	13
9.3 Operating system simulation	13
9.4 Hardware prototyping	13
10. Conclusion and Future scope	14
References	15
Annexure	

List of Figures

Fig no.	Figure name	Page no.
Fig. 4.1	Block diagram	7
Fig. 6.1	Flowchart	9
Fig. 8.1	Output using Synopsys	11
Fig. 8.2	Output using EDA playground	11

CHAPTER – 1

INTRODUCTION

Memory management is a critical component of computer architecture, ensuring efficient access to data by controlling how the system's memory hierarchy is utilized. One of the core aspects of memory management is the cache system, which is designed to provide high-speed access to frequently used data, reducing the time it takes to fetch data from main memory. This project focuses on the design and implementation of a Memory Management System (MMS) with a Direct Mapped Cache and Write-Through Policy using verilog. The purpose of this design is to demonstrate how cache memory can be managed effectively within a system, optimizing performance for memory access. The key features of this system include:

1. **Direct Mapped Cache:** This cache architecture maps each memory block to exactly one cache line, which simplifies the design and implementation. The cache lookup involves checking if the data corresponding to the memory address is present in the cache (cache hit) or needs to be fetched from the main memory (cache miss).
2. **Write-Through Policy:** The write-through policy ensures that any write operation to the cache is simultaneously written to the main memory. This simplifies the consistency between cache and memory but can result in higher memory traffic.
3. **Verilog Implementation:** Verilog, a hardware description language (HDL), will be used to design the cache system. The primary goal is to create a simple, yet effective, implementation that highlights the fundamental concepts of cache management without the complexity of more advanced techniques like associative mapping or write-back policies.

The design of the memory management system will include components such as:

Cache memory organization

Address decoding and mapping mechanisms

Write-through handling for memory operations

Verilog modules for cache and memory interaction

By implementing this design, we aim to demonstrate the efficiency and simplicity of a direct-mapped cache with a write-through policy, providing a foundation for more complex cache architectures and memory management techniques. This project also serves as a practical exercise in Verilog, offering insights into digital design and hardware simulation for memory subsystems.

1.1 Objective:

To design a memory management system featuring a direct-mapped cache and a write-through policy.

To implement this system using Verilog, targeting basic hardware design principles and efficient memory access strategies.

1.2 Research Methodology:

1. Problem Identification and Requirement Analysis: Understand the limitations in basic memory access mechanisms and the need for efficient cache management.

Tasks: Study the principles of computer memory hierarchy.

Identify performance bottlenecks in systems without cache.

2. Literature Review: Explore existing cache architectures, memory access policies, and hardware implementation practices.

Tasks: Review academic papers, textbooks, and digital design documents on cache memory system

Compare different cache mapping techniques (direct-mapped, fully associative, set-associative).

3. System Design: Create a conceptual and architectural model of the memory management System.

Tasks: Design the structure of the cache (tag, index, data, valid bit).

Determine how memory addresses will be partitioned (tag, index, offset).

4. Verilog Implementation: Translate the design into Verilog modules for simulation.

Tasks: Write Verilog modules.

Implement finite state machines (FSMs) for cache operations.

5. Simulation and Testing: Validate the functionality and correctness of the Verilog implementation.

Tasks: Create testbenches in Verilog to simulate various memory access patterns.

Simulate cache hit and miss scenarios.

6. Performance Evaluation: Analyse the effectiveness of the implemented system.

Tasks: Measure cache hit rate and memory access latency.

Compare performance with and without cache.

7. Documentation and Analysis: Present findings and provide insights for future improvements.

Tasks: Document design decisions, simulation results, and performance metrics.

Identify limitations in the current implementation.

CHAPTER – 2

Literature Survey

2.1 Memory Hierarchy and Cache Fundamentals:

Hennessy, J. L., & Patterson, D. A. (Computer Architecture: A Quantitative Approach)
This seminal textbook outlines the memory hierarchy model and explains how cache systems enhance performance by reducing access latency. It introduces various cache mapping techniques such as direct-mapped, set-associative, and fully-associative, highlighting the simplicity and speed of direct-mapped caches. It also contrasts write-through and write-back policies, noting that while write-through is slower due to frequent memory writes, it offers better data consistency.

Tanenbaum, A. S. (Structured Computer Organization)
Emphasizes the importance of caching mechanisms in computer systems, including how direct mapping reduces the complexity of cache controllers. The book also provides a basic understanding of how address translation and cache tagging work in such architectures.

2.2 Cache Mapping Techniques:

Research Article: "A Comparative Study of Cache Mapping Techniques in Memory Management"

This study compares direct-mapped, fully associative, and set-associative caches. It finds that while fully associative caches offer higher hit rates, direct-mapped caches offer the best trade-off between speed and implementation simplicity—making them suitable for low-complexity hardware designs, especially for FPGA-based systems.

IEEE Conference Paper: "Design and Implementation of Cache Memory on FPGA"

This paper explores direct mapped cache implementation using VHDL and Verilog. It validates the efficiency of FPGA-based cache models and emphasizes the low hardware overhead and fast response time of direct mapping schemes.

2.3 Write Policies in Cache:

Research Article: "Analysis of Write Policies in Cache Memory Systems"

This paper explains the operational differences between write-through and write-back policies. Write-through is easier to implement and ensures data consistency between cache and memory, which is critical in real-time systems, despite its performance trade-offs due to higher memory traffic.

Textbook Reference: "Digital Design and Computer Architecture" by Harris & Harris

The book elaborates on the practical implications of different cache write policies, showing examples in hardware design where write-through is preferred for its predictability and simplicity.

2.4 Verilog HDL for Memory Systems:

Verilog HDL by Samir Palnitkar

Offers a foundational understanding of using Verilog to model hardware components. The book includes examples of memory modules and finite state machines, which are crucial for designing the control logic of a cache system.

Research Paper: "Cache Memory Design Using Verilog and Implementation on FPGA"

This paper presents a practical approach to implementing cache memory on FPGA using Verilog. It discusses cache controller design, cache hit/miss handling, and memory interface, providing insights into simulation and testing strategies.

2.5 Simulation and Evaluation Techniques:

IEEE Journal: "Evaluation Metrics for Cache System Simulation"

This article describes the standard performance metrics used in evaluating cache memory, including hit rate, miss penalty, and memory access time. It supports the methodology of testing the cache design under different access patterns and load conditions.

Research Study: "Performance Analysis of Cache Architectures for Embedded Systems"

Focuses on cache design in resource-constrained environments. The study supports using direct-mapped caches and simple write policies in embedded systems where deterministic performance and ease of implementation are essential.

CHAPTER – 3

Research Gap

Despite extensive studies on memory hierarchy and cache architectures in modern computer systems, most existing research and educational resources focus on complex and performance-optimized cache systems such as set-associative and fully associative caches, or multi-level caching. These implementations often involve sophisticated techniques like replacement policies and write-back mechanisms, which, while powerful, increase design complexity—especially for learners or for use in small-scale embedded systems.

However, this is a notable lack of practical, educational-level projects that demonstrate:

The use of direct mapped cache structures, which are simpler and easier to understand.

Implementation of the write-through policy, which, despite being less efficient in terms of memory traffic, offers better consistency and is easier to design.

A basic verilog-based suitable for simulation, teaching, and prototyping purposes, particularly for FPGA or introductory digital design courses.

Furthermore, the integration of all these elements—a direct mapped cache, write-through consistency policy, and verilog—into a cohesive, well-documented system is not widely covered in current literature or open educational repositories.

CHAPTER – 4

Block Diagram

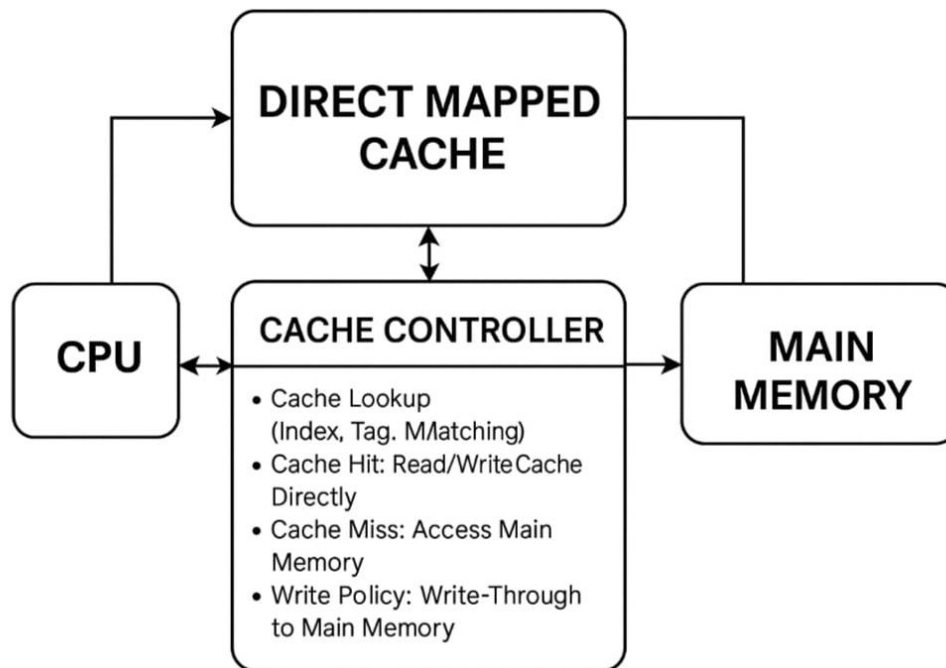


Fig. 4.1

This block diagram illustrates how a Direct Mapped Cache system works in conjunction with the CPU, Main Memory, and Cache Controller.

4.1 Flow Summary:

1. CPU sends a data request to the Cache Controller.
2. Cache Controller checks the Direct Mapped Cache using index and tag.
3. If data is present (hit), it is sent to CPU.
4. If data is absent (miss), it is fetched from Main Memory, possibly updating the cache.
5. For writes, data is written to both cache and memory (write-through policy).

CHAPTER – 5

Methodology

The system consists of four primary components: the CPU, the Direct Mapped Cache, the Main Memory and the Cache Controller. The CPU acts as the processor that issues memory access requests—both read and write. The cache, which is much smaller and faster than main memory, is used to store a subset of data temporarily to reduce access times. The cache controller acts as the mediator, handling cache lookups, determining cache hits or misses, and enforcing read/write policies. The main memory stores all data permanently and is accessed only when the required data is not available in the cache. In a Direct Mapped Cache, each memory block from main memory maps to exactly one cache line. The mapping is simple and efficient, using portions of the memory address. The memory address is logically divided into three parts: the tag, index, and block offset. The index selects the specific cache line, while the tag is used to confirm whether the data in the selected cache line is indeed the desired data. The offset helps identify the specific word within the cache block. When the CPU generates a memory access request, the cache controller first performs a cache lookup using the index and tag derived from the address. If the tag in the cache matches the tag from the CPU's address and the cache line is valid, it results in a cache hit, allowing data to be read or written directly in the cache. If the tags do not match or the valid bit is not set, it results in a cache miss, and the data must be fetched from main memory. Once retrieved, the data is placed into the appropriate cache line, and the tag and valid bit are updated accordingly.

The system follows a write-through policy for write operations, where any data written to the cache is simultaneously written to main memory. This ensures consistency between cache and memory, although it may introduce additional latency due to memory write operations. Performance can be evaluated in terms of the cache hit ratio, which is the fraction of memory accesses that result in a cache hit. A higher hit ratio indicates more efficient cache utilization and improved overall system performance. This methodology assumes a fixed block size for cache and main memory, with aligned memory addresses for simplicity.

CHAPTER – 6

Flowchart/Algorithm

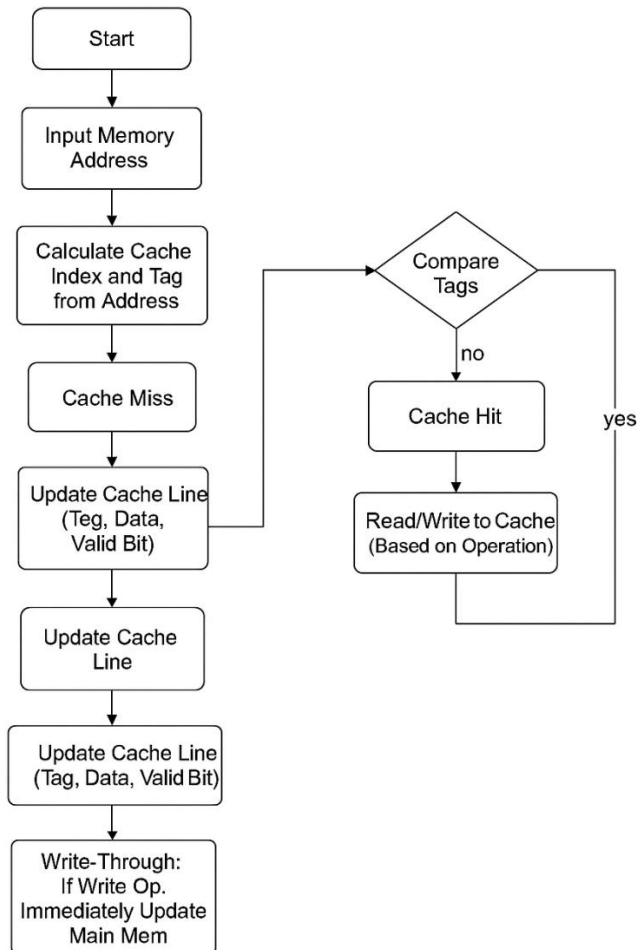


Fig. 6.1

This flowchart illustrates the working of a cache memory system with direct mapped cache and write-through policy. The process begins when a memory address is provided as input, typically from a processor requesting data. The system first calculates the cache index and tag from the given address by dividing the address into tag, index, and offset bits. It then checks if the tag in the cache at the specified index matches the tag from the input address. If the tags match and the valid bit is set, it indicates a cache hit, allowing the system to directly read from or write to the cache depending on the operation. If there is no match, it results in a cache miss. In such cases, the cache line is updated with the correct tag, the retrieved data from main memory, and the valid bit is set. For write operations, the system uses a write-through approach, meaning it writes data to both the cache and the main memory simultaneously to maintain consistency. This approach ensures that the main memory always reflects the most recent data, even if the cache is updated. Overall, the flowchart captures the basic logic of a direct-mapped cache system with immediate memory synchronization through the write-through policy.

CHAPTER – 7

Software used

1. EDA playground: EDA Playground is a popular web-based platform used for simulating and testing digital design code in Hardware Description Languages (HDLs) like Verilog, System Verilog, and VHDL. It is widely used by students, educators, and professionals for learning, teaching, and quick prototyping of digital logic designs.

7.1 Key Features:

2. Online Simulation: EDA Playground allows users to write, simulate, and debug HDL code directly in the browser without needing to install any software. It integrates with several industry-standard simulators like ModelSim, Synopsys VCS, and Cadence Incisive, depending on the licensing and availability.
3. Multi-language Support: It supports both Verilog and VHDL, as well as SystemVerilog, making it a versatile environment for a variety of digital logic and design projects.
4. Code Sharing: One of the most valuable features is the ability to share code using a simple URL. This is useful for collaboration, code review, and academic purposes where instructors can provide students with example code or assignments.
5. Testbenches and Simulation Outputs: Users can create testbenches to simulate their HDL designs. The simulation results, such as waveform outputs or console logs, are easily viewable within the platform.
6. Parameter and Tool Options: EDA Playground allows customization of simulation settings such as time units, optimizations, and tool-specific options.
7. Learning Resource: It serves as a great learning tool for beginners in digital design, especially those practicing for hardware engineering interviews, university assignments, or competitive VLSI courses.

CHAPTER – 8

Result analysis

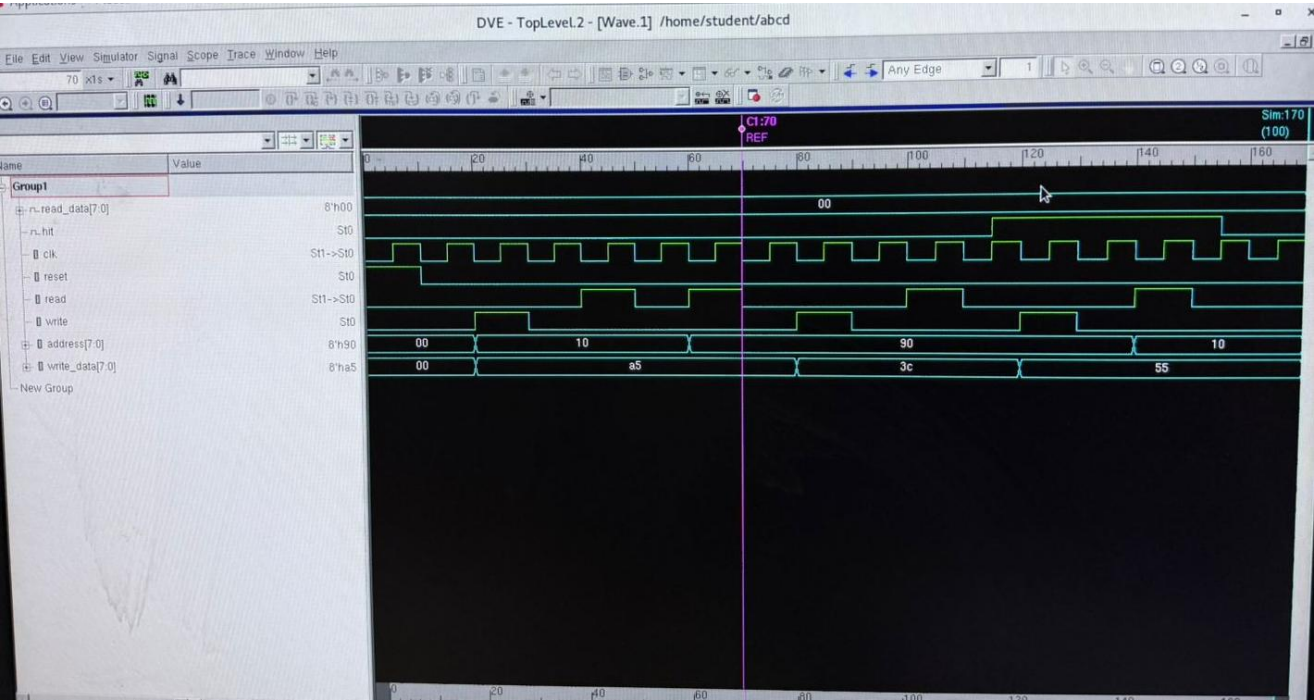


Fig. 8.1

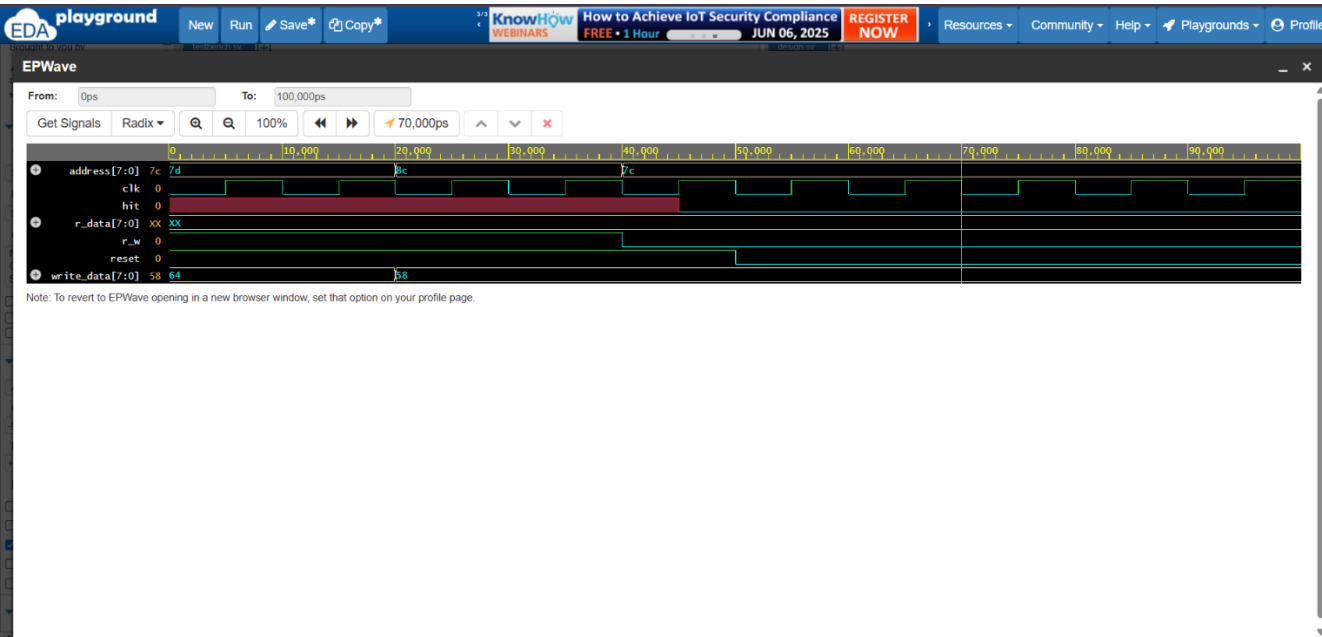


Fig.8.2

Reset Phase: $\text{reset} = 1$: The module is inactive. All internal registers are typically cleared or set to known values. Once $\text{reset} = 0$, the module is ready to function.

Write Operation: $r_w = 0$ signals a write that is data is written. `address` holds the target location. `write_data` contains the data to write. After a few clock cycles, the module completes the write. $\text{hit} = 1$ is set to indicate the operation succeeded.

Read Operation: $r_w = 1$ shows that the data is to be read. `address` holds the location to be read from. After some clock cycles, `r_data` becomes valid. $\text{hit} = 1$ indicates the data is ready on `r_data`.

CHAPTER – 9

Applications

9.1 Computer Architecture Education

This project serves as a practical educational tool for teaching memory hierarchy concepts in computer architecture. By implementing a direct mapped cache and write-through policy, students can clearly understand how data is accessed, cached, and synchronized with main memory.

9.2 Embedded Systems Design

In embedded systems where hardware resources are limited and performance is critical, a direct mapped cache with a write-through policy offers a simple yet efficient memory management solution. This system ensures consistency between cache and memory, which is crucial for real-time applications where predictability and reliability of data are more important than high-speed performance.

9.3 Operating System Simulation

The design can be used to simulate a simplified version of memory management found in operating systems. It helps learners and researchers explore how memory access patterns affect cache performance and how policies like write-through impact system behaviour. This is particularly useful in academic projects and research that study memory management algorithms.

9.4 Hardware Prototyping

It allows students and engineers to experiment with cache design in hardware, test latency, and verify system behaviour under real-time constraints. It's an ideal foundation for more advanced system-on-chip (SOC) designs.

CHAPTER – 10

Conclusion and Future Scope

Conclusion:

The implementation of a memory management system using a direct mapped cache with a write-through policy demonstrates a fundamental and effective approach to improving memory access speed while maintaining data consistency between the cache and main memory. By simulating read and write operations, cache hits and misses, and write-through behaviour, the system provides valuable insights into the inner workings of memory hierarchy in computing systems. Overall, the project serves as a fast and better way to read and write data.

Future scope:

1. **Implementation of Set-Associative and Fully Associative Cache:** Future work can involve extending the system to support more advanced cache mapping techniques such as set-associative or fully associative caches. This would allow comparison of performance metrics and a deeper understanding of cache efficiency.
2. **Write-Back Policy Integration:** Incorporating the write-back policy along with dirty bit management would provide a more comprehensive view of cache memory design, allowing exploration of latency and write efficiency in more complex systems.
3. **Multi-Level Cache Hierarchy (L1, L2):** The design can be expanded to include multi-level caches (L1, L2) to reflect modern processor architecture, which would help in simulating realistic memory access scenarios and analysing the impact on performance.
4. **Integration with CPU Simulation:** Integrating the memory system with a basic CPU simulator can simulate instruction-level memory accesses and demonstrate the interaction between processing and memory subsystems.

REFERENCES

1. Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design: The Hardware/Software Interface* (5th ed.). Morgan Kaufmann.
2. Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
3. Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
4. Jacob, B., Ng, S., & Wang, D. (2007). *Memory Systems: Cache, DRAM, Disk*. Morgan Kaufmann.
5. Author(s) Unknown. (Year Unknown). *A Study of Cache Performance for Write-Through and Write-Back Policies*. IEEE Xplore. Retrieved from <https://ieeexplore.ieee.org>
6. GeeksforGeeks. (n.d.). Cache Memory in Computer Organization. Retrieved May 2025, from <https://www.geeksforgeeks.org/cache-memory-in-computer-organization/>
7. TutorialsPoint. (n.d.). Cache Memory. Retrieved May 2025, from https://www.tutorialspoint.com/computer_fundamentals/cache_memory.htm
8. Massachusetts Institute of Technology. (2018). *Computer System Engineering (6.033)*. MIT OpenCourseWare. Retrieved from <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-033-computer-system-engineering-spring-2018/>

Annexure

Verilog design code:

```
module cache_controller(  
    input clk,  
    input reset,  
    input r_w,  
    input [7:0] address,  
    input [7:0] write_data,  
    output reg [7:0] read_data,  
    output reg hit  
);  
    reg [7:0] cache_data [0:15];  
    reg [3:0] cache_tag [0:15];  
    reg cache_valid [0:15];  
    reg [3:0] idx,tg;  
    reg [0:7] main_memory [0:255];  
    reg [7:0] s_addr,s_data;  
  
    parameter ideal = 2'b00, write = 2'b01, read_cache=2'b10, read_memory=2'b11;  
  
    reg [1:0] pres_state = ideal, next_state = ideal;  
  
    always@(posedge clk)
```

```
begin

case(pres_state)

ideal:

begin

if(reset & r_w)

begin

next_state = write;

s_addr = address;

s_data = write_data;

end

else if (reset & !r_w)

begin

next_state = read_cache;

s_addr = address;

hit = 0;

end

else

next_state = ideal;

end

end

write:

begin

cache_data[s_addr[3:0]] = s_data;
```



```
    cache_tag[s_addr[3:0]] = s_addr[7:4];  
  
    cache_valid[s_addr[3:0]] = 1'b1;  
  
    main_memory[s_addr] = s_data;  
  
    next_state = ideal;  
  
end
```

```
read_cache:
```

```
begin  
  
    idx = s_addr[3:0];  
  
    tg = s_addr[7:4];  
  
  
    if (cache_valid[idx] && cache_tag[idx] == tg)  
  
        begin  
  
            hit = 1;  
  
            read_data = cache_data[idx];  
  
            next_state = ideal;  
  
        end  
  
    else  
  
        begin  
  
            hit = 0;  
  
            next_state = read_memory;  
  
        end  
  
end
```

```
read_memory:
    begin
        read_data = main_memory[s_addr];
        cache_data[s_addr[3:0]] = main_memory[s_addr];
        cache_tag[s_addr[3:0]] = s_addr[7:4];
        next_state = ideal;
    end
endcase
pres_state = next_state;
end
endmodule
```

Testbench:

```
module tb;

    reg clk=0;

    reg reset;

    reg r_w;

    reg [7:0] address;

    reg [7:0] write_data;

    reg hit;

    reg [7:0] r_data;

    cache_controller
```

```
DUT(.clk(clk),.reset(reset),.r_w(r_w),.address(address),.write_data(write_data),.hit(hit),.read_data(r_data));
```

```
always #5 clk = ~clk;
```

```
initial begin
```

```
    $dumpfiles("test.vcd");
```

```
    $dumpvars();
```

```
    reset = 1;
```

```
    r_w = 1;
```

```
    address = 8'd125;
```

```
    write_data = 8'd100;
```

```
    #20;
```

```
    address = 8'd140;
```

```
    write_data = 8'd88;
```

```
    #20;
```

```
    r_w = 0;
```

```
    address = 8'd124;
```

```
    #10;
```

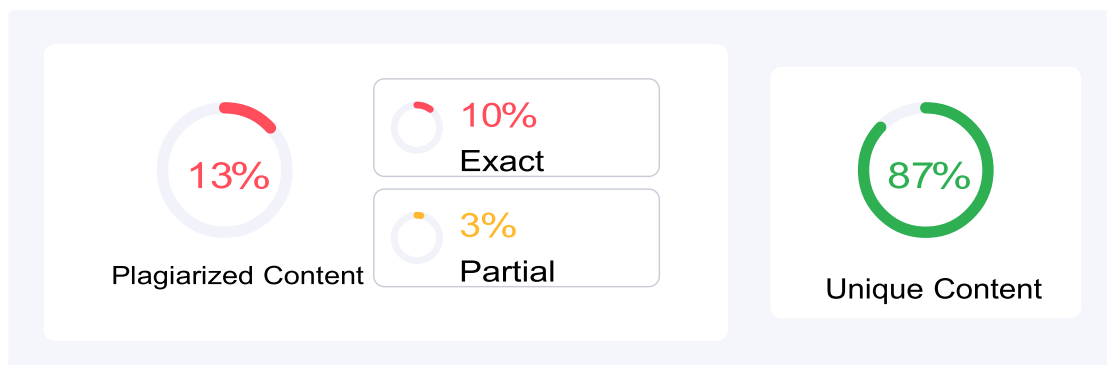
```
    reset = 0;
```

```
end
```

```
endmodule
```

Plagiarism Scan Report By SmallSEOTools

Report Generated on: Mav



Total Words1000

Total Characters6913

Plagiarized Sentences52

Unique Sentences64.8 (87%)

Content Checked for Plagiarism

JNANA SANGAMA, BELAGAVI-590015, KARNATAKA Project Report on DESIGN AND IMPLEMENTATION OF MEMORY MANAGEMENT WITH DIRECT MAPPED CACHE AND WRITE

THROUGH POLICY Submitted in partial fulfillment of the requirements for the IV semester Mini Project

Work [SoC Design &Flow] [EC272] Bachelor of Engineering in Electronics and Communication Engineering of Visvesvaraya Technological University, Belagavi. by Charvanya H L (1CD23EC028) Bhavana M V (1CD23EC019) Isheeta Pandey (1CD23EC062) Aishwarya Sonakar (1CD23EC007)

Under the Guidance of Mr. Umesh Royal External Guide Dr. Indu K. Assistant Professor Elevium-by Nanochip ECE, CIT Department of Electronics and Communication Engineering CAMBRIDGE INSTITUTE OF TECHNOLOGY, BANGALORE - 560 036 2024-2025 CAMBRIDGE INSTITUTE OF

TECHNOLOGY K.R. Puram, Bangalore-560 036 DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING CERTIFICATE It is Certified that Ms. Charvanya H L, Ms. Bhavana M

V, Ms. Isheeta Pandey and Ms. Aishwarya Sonakar bearing USN: 1CD23EC028, 1CD23EC019, 1CD23EC062 and 1CD23EC007 respectively, are bona fide students of Cambridge Institute of Technology and have completed requirements of the project entitled —Design and implementation of memory management system with direct mapped cache and write through policy in partial fulfillment of the requirements for IV semester Bachelor of Engineering in Electronics and Communication Engineering of Visvesvaraya Technological University, Belagavi during academic year 2024-25. It is certified that all Corrections/Suggestions indicated for Internal Assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of mini project prescribed for the Bachelor of Engineering degree. Mr. Umesh Royal External Guide, Elevium Dr. Indu K. Internal Guide, ECE Dept- CITech Dr. Shivapanchakshari T.G. Prof. and HOD ECE,

CITech Dr. G. Indumathi, Principal, CITech External Viva: Name of the Examiners Signature with Date
1. 2. DECLARATION We, Ms. Charvanya H L, Ms. Bhavana M V, Ms. Isheeta Pandey and Ms. Aishwarya

Sonakar bearing USN: 1CD23EC028, 1CD23EC019, 1CD23EC062 and 1CD23EC007 respectively, are students of IV semester, Electronics and Communication Engineering, Cambridge Institute of Technology, hereby declare that the project entitled —Design and implementation of memory management system with direct mapped cache and write through policy has been carried out by us and submitted in partial fulfillment of the course requirements MINI PROJECT of IV semester Bachelor of Engineering in Electronics and Communication Engineering as prescribed by Visvesvaraya Technological University, Belagavi, during the academic year 2024-2025. We also declare that, to the best of our knowledge and belief, the work reported here does not form part of any other report on the basis of which a degree or award was conferred on an earlier occasion on this by any other student.

Date: 14/05/2025 Place: Bengaluru Charvanya H L (1CD23EC028) Bhavana M V (1CD23EC019)

Isheeta Pandey (1CD23EC062) Aishwarya Sonakar (1CD23EC007) ACKNOWLEDGEMENT I would like to place on record my deep sense of gratitude to Shri D. K. MOHAN, Chairman, Cambridge Group of Institutions, Bangalore, India for providing excellent Infrastructure and Academic Environment at

CITECH without which this work would not have been possible. I extremely thankful to Dr. G. INDUMATHI, PRINCIPAL, Cambridge Institute of Technology, Bengaluru for providing me Academic ambience and Laboratory facilities to in and everlasting motivation to carry out this work and shaping our careers. I express my sincere gratitude to Dr. SHIVAPANCHAKSHARI T. G. HOD, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for his stimulating guidance, continuous encouragement and motivation throughout the course of present work. I also wish to extend my thanks to our Internal Guide Dr. Indu K. Assistant Professor,

Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for their unstilted support, valuable guidance and help throughout the work. I also wish to extend my thanks to our External Guide Mr. Umesh Royal, Elevium, Bengaluru for her critical, insightful comments, guidance and constructive suggestion and support to improve the quality of this project work. I would like to thank all the faculty members and non-teaching staff of Dept. of ECE and Elevium team for their constant support. And I would like to thank our parents and friends for their constant moral and financial support. Charvanya H L (1CD23EC028) Bhavana M V (1CD23EC019) Isheeta Pandey (1CD23EC062) Aishwarya Sonakar (1CD23EC007) ABSTRACT efficient memory management is a critical aspect of modern computing systems, directly in using system performance, power efficiency, and responsiveness. Our project focuses on the design and implementation of a simplified memory management system featuring a direct-mapped cache architecture with a write-through policy, developed using verilog hardware description language. The goal is to simulate a small-scale yet functional model of cache memory behaviour to demonstrate fundamental concepts in computer architecture and digital system design. The system is designed to operate as an intermediary between a CPU and main memory, providing faster access to frequently used data and instructions. A direct-mapped cache structure is selected for its simplicity and speed. In this mapping scheme, each block of main memory maps to exactly one cache line, reducing the complexity of address comparison logic and ensuring a deterministic mapping. Although it has limitations in terms of conflict misses, its straightforward structure is ideal for educational and foundational design purposes. The write-through

policy ensures that every write operation to the cache also updates the corresponding location in the main memory. This policy simply consistency management between cache and memory, as there is no risk of data becoming stale. While writethrough introduces a performance trade-off due to more frequent memory writes, it eliminates the need for complex write- back logic and improves reliability in systems where data integrity is paramount. The verilog implementation includes modules for the cache controller, tag comparator, address decoder, memory interface, and basic nite state machines (FSMs) for read and write operations. The system simulates how memory requests are handled: on a cache hit, data is retrieved from or written to the cache directly; on a cache miss, data is fetched from the main memory, and the cache is updated accordingly. The design also supports a simple testbench to simulate various access patterns and validate cache behaviour under different scenarios. Table of Contents
Contents 1. Introduction 2. Literature Survey 3. Research

Plagiarized Sources

It is certified that all Corrections/Suggestions indicated for Internal Assessment have been incorporated in the report. [↗](#)

<https://www.studocu.com/in/document/sir-mokshagundam-visvesvaraya-institute-of-technology/automatatheory-of-computation/aicte-nal-report-repaired/31235136>

MOHAN, Chairman, Cambridge Group of Institutions, Bangalore, India for providing excellent Infrastructure and Academic Environment at CITECH without which this work would not have been possible. [↗](#)

<https://www.studocu.com/in/document/cambridge-group-of-institutions/computer-science/seminarreport/94056091>

